

BIBLE MOVIEPY – RÉFÉRENCE EXHAUSTIVE DU MODULE

LÉGENDE : [vc] = VideoClip | [vfc] = VideoFileClip | [ac] = AudioClip
[afc] = AudioFileClip | [fx] = effets | [comp] = CompositeVideoClip

[vfc] Charger une vidéo – VideoFileClip

DESCRIPTION : Charger un fichier vidéo existant pour le manipuler.
POURQUOI : Point d'entrée principal pour éditer des vidéos existantes.
CONTEXTE : MP4, AVI, MOV, MKV, WebM, GIF, et tout format supporté par FFmpeg.

— CHARGEMENT —

```
from moviepy import VideoFileClip
```

clip = VideoFileClip("video.mp4")	Charge une vidéo
clip = VideoFileClip("video.mp4", audio=True)	Avec audio (défaut)
clip = VideoFileClip("video.mp4", audio=False)	Sans audio
clip = VideoFileClip("video.mp4", target_resolution=(720, 1280))	Redimensionne au chargement (h, w)
clip = VideoFileClip("video.mp4", resize_algorithm="bilinear")	Algorithme de redimensionnement
clip = VideoFileClip("video.mp4", fps_source="fps")	Source du FPS ("fps" ou "tbr")

— GESTIONNAIRE DE CONTEXTE —

```
with VideoFileClip("video.mp4") as clip:
    print(clip.duration)
```

Ferme proprement les ressources

— ATTRIBUTS PRINCIPAUX —

clip.duration	Durée en secondes (float)
clip.fps	Images par seconde
clip.size	Dimensions (largeur, hauteur) en pixels
clip.w / clip.width	Largeur en pixels
clip.h / clip.height	Hauteur en pixels
clip.aspect_ratio	Ratio largeur/hauteur
clip.audio	Piste audio (AudioClip ou None)
clip.start	Temps de départ (dans la timeline)
clip.end	Temps de fin
clip.filename	Chemin du fichier source
clip.nframes	Nombre total de frames

EXEMPLES :

```
clip = VideoFileClip("film.mp4")
print(f"Durée : {clip.duration:.1f}s | {clip.fps}fps | {clip.size}")
clip.close()
```

[vc] Découpe et sous-clips – subclipped / with_start / with_end

DESCRIPTION : Extraire une portion temporelle d'un clip.
POURQUOI : Couper le début, la fin, ou un segment précis.
CONTEXTE : Montage, extraction de highlights, découpe de segments.

FONCTIONS PRINCIPALES :

clip.subclipped(0, 10)	Secondes 0 à 10
clip.subclipped(30)	De 30s jusqu'à la fin
clip.subclipped(0, -5)	Tout sauf les 5 dernières secondes
clip.subclipped("00:01:00", "00:02:30")	Format hh:mm:ss
clip.subclipped(t_start=10, t_end=20)	Avec arguments nommés
clip.with_start(5)	Définit le début dans la timeline
clip.with_end(30)	Définit la fin dans la timeline
clip.with_duration(10)	Définit la durée

EXEMPLES :

```
# Extraire les 30 premières secondes
intro = clip.subclipped(0, 30)

# Supprimer les 5 premières et dernières secondes
propre = clip.subclipped(5, clip.duration - 5)
```

```
# Découpe par timecode
scene = clip.subclipped("00:01:15", "00:02:45")
```

[vc] Redimensionnement et recadrage – resized / cropped

DESCRIPTION : Changer la taille ou rogner une vidéo.
 POURQUOI : Adapter à une résolution cible, créer du 9:16 depuis du 16:9.
 CONTEXTE : Export réseaux sociaux, optimisation de taille, recadrage centré.

— REDIMENSIONNEMENT —

<code>clip.resized((1280, 720))</code>	Taille exacte (largeur, hauteur)
<code>clip.resized(width=1280)</code>	Largeur fixe, hauteur auto
<code>clip.resized(height=720)</code>	Hauteur fixe, largeur auto
<code>clip.resized(0.5)</code>	50% de la taille d'origine
<code>clip.resized(2.0)</code>	Doubler la taille
<code>clip.resized(new_size=(640, 480))</code>	Avec argument nommé

— RECADRAGE —

<code>clip.cropped(x1=100, x2=500, y1=50, y2=400)</code>	Recadre par coordonnées pixels
<code>clip.cropped(x_center=640, y_center=360, width=1280, height=720)</code>	Recadre centré
<code>clip.cropped(width=720, height=1280, x_center=clip.w/2, y_center=clip.h/2)</code>	Format portrait depuis paysage

EXEMPLES :

```
# Créer un format 9:16 (TikTok/Reels) depuis du 16:9
```

```
portrait = clip.cropped(
    x_center=clip.w / 2,
    width=clip.h * 9 / 16,
    height=clip.h
)
```

```
# Réduire pour l'export web
web = clip.resized(width=1280)
```

[vc] Rotation et retournement – rotated / with_effects

DESCRIPTION : Faire pivoter ou retourner un clip.

FONCTIONS PRINCIPALES :

<code>clip.rotated(90)</code>	Rotation 90° (sens antihoraire)
<code>clip.rotated(180)</code>	Retournement vertical
<code>clip.rotated(-90)</code>	Rotation 90° horaire
<code>clip.rotated(45, expand=True)</code>	Rotation libre avec expansion du cadre
<code>clip.rotated(45, expand=False)</code>	Rotation libre sans expansion (recadré)

```
from moviepy.video.fx import MirrorX, MirrorY
clip.with_effects([MirrorX()])    Miroir horizontal
clip.with_effects([MirrorY()])    Miroir vertical
```

[vc] Vitesse et temps – with_multiply_speed / time_mirror

DESCRIPTION : Modifier la vitesse de lecture d'un clip.
 POURQUOI : Slow motion, time-lapse, effets dramatiques.

FONCTIONS PRINCIPALES :

```
from moviepy.video.fx import MultiplySpeed, TimeMirror, Freeze
```

<code>clip.with_effects([MultiplySpeed(2)])</code>	Double la vitesse (x2)
<code>clip.with_effects([MultiplySpeed(0.5)])</code>	Ralenti à 50% (slow motion)
<code>clip.with_effects([MultiplySpeed(factor=4)])</code>	Time-lapse x4
<code>clip.with_effects([TimeMirror()])</code>	Lecture à l'envers
<code>clip.with_effects([Freeze(t="00:00:05", freeze_duration=2)])</code>	Figé une frame Pendant 2 secondes

```

EXAMPLES      :
# Slow motion x4
slow = clip.with_effects([MultiplySpeed(0.25)])

# Boomerang (avant + arrière)
from moviepy import concatenate_videoclips
from moviepy.video.fx import TimeMirror
boomerang = concatenate_videoclips([clip, clip.with_effects([TimeMirror()])])

```

[vc] Effets visuels – with_effects / fx

DESCRIPTION : Appliquer des filtres et effets visuels aux clips.
POURQUOI : Transitions, corrections colorimétriques, effets créatifs.

— LUMINOSITÉ / COULEUR —

```

from moviepy.video.fx import (
    BlackAndWhite, InvertColors, GammaCorrection,
    LumContrast, MakeLoopable, Normalize,
    ColorMatrixSwap, MultiplyColor,
)

clip.with_effects([BlackAndWhite()])      Noir et blanc
clip.with_effects([InvertColors()])      Inversion des couleurs
clip.with_effects([GammaCorrection(gamma=1.5)]) Correction gamma
clip.with_effects([LumContrast(lum=10,
                               contrast=0.5)]) Luminosité + contraste
clip.with_effects([Normalize()])         Normalise les niveaux
clip.with_effects([MultiplyColor(factor)]) Multiplie une couleur

```

— FLOU ET DÉFORMATION —

```

from moviepy.video.fx import Painting, Pixelize

clip.with_effects([Pixelize(pixel_size=10)]) Pixelisation

```

— TRANSPARENCE / COMPOSITION —

```

from moviepy.video.fx import FadeIn, FadeOut, CrossFadeIn, CrossFadeOut

clip.with_effects([FadeIn(duration=1)])      Fondu d'entrée (1 seconde)
clip.with_effects([FadeOut(duration=1)])     Fondu de sortie
clip.with_effects([CrossFadeIn(duration=1)]) Fondu enchaîné entrée
clip.with_effects([CrossFadeOut(duration=1)]) Fondu enchaîné sortie

```

— ENCHAÎNER PLUSIEURS EFFETS —

```

clip.with_effects([
    BlackAndWhite(),
    FadeIn(duration=0.5),
    FadeOut(duration=0.5),
])

```

— EFFET PERSONNALISÉ —

```

# Appliquer une fonction frame par frame
import numpy as np

def sepia(frame):
    sepia_matrix = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131],
    ])
    return np.clip(frame @ sepia_matrix.T, 0, 255).astype("uint8")

clip_sepia = clip.image_transform(sepia)

# Transformation avec le temps
def zoom_in(get_frame, t):
    frame = get_frame(t)
    zoom = 1 + 0.1 * t
    h, w = frame.shape[:2]
    new_h, new_w = int(h / zoom), int(w / zoom)
    y, x = (h - new_h) // 2, (w - new_w) // 2
    return frame[y:y+new_h, x:x+new_w]

clip_zoom = clip.transform(zoom_in)

```

[comp] Composition de clips – CompositeVideoClip

DESCRIPTION : Superposer plusieurs clips en couches (comme dans un éditeur vidéo).
POURQUOI : Incrustations, PIP (picture-in-picture), texte en surimpression.
CONTEXTE : Watermarks, sous-titres, logo, split-screen.

— SUPERPOSITION —

```
from moviepy import CompositeVideoClip

final = CompositeVideoClip([fond, overlay])  fond en bas, overlay au-dessus
final = CompositeVideoClip([fond, overlay],
    size=(1920, 1080))                        Taille explicite de la composition
final = CompositeVideoClip([fond, overlay],
    use_bgclip=True)                          Utilise le premier clip comme fond
```

— POSITIONNEMENT —

```
overlay.with_position((100, 50))             Position absolue (x, y) en pixels
overlay.with_position("center")               Centré
overlay.with_position(("center", "top"))      Centré horizontalement, en haut
overlay.with_position(("right", "bottom"))    En bas à droite
overlay.with_position(("left", "center"))     À gauche, centré verticalement
overlay.with_position(lambda t: (100+t*10, 50)) Position animée
```

— TIMING DES CALQUES —

```
overlay.with_start(3)                        Apparaît à 3s
overlay.with_end(8)                          Disparaît à 8s
overlay.with_start(3).with_end(8)            Apparaît entre 3 et 8s
```

EXEMPLES :

```
# Picture-in-picture
main = VideoFileClip("main.mp4")
pip = VideoFileClip("cam.mp4").resized(0.25)
pip = pip.with_position(("right", "top")).with_start(0)

final = CompositeVideoClip([main, pip])
final.write_videofile("pip.mp4", fps=main.fps)

# Watermark image
from moviepy import ImageClip
logo = (ImageClip("logo.png")
    .with_duration(main.duration)
    .resized(height=50)
    .with_position(("right", "bottom"))
    .with_opacity(0.5))

final = CompositeVideoClip([main, logo])
```

[vc] Concaténation – concatenate_videoclips

DESCRIPTION : Mettre des clips bout à bout.
POURQUOI : Montage linéaire, assemblage de séquences.

FONCTIONS PRINCIPALES :

```
from moviepy import concatenate_videoclips

final = concatenate_videoclips([clip1, clip2, clip3])
final = concatenate_videoclips([clip1, clip2],
    method="compose")                        Compose si tailles différentes
final = concatenate_videoclips([clip1, clip2],
    method="chain")                          Enchaîne sans recomposer (même taille requise)
final = concatenate_videoclips([clip1, clip2],
    padding=-0.5)                            Superposition de 0.5s (transition)
final = concatenate_videoclips([clip1, clip2],
    transition=None)                         Pas de transition
```

EXEMPLES :

```
clips = [VideoFileClip(f) for f in ["a.mp4", "b.mp4", "c.mp4"]]
final = concatenate_videoclips(clips, method="compose")
final.write_videofile("montage.mp4", fps=30)
for c in clips:
```

```

c.close()

# Avec fondu enchaîné entre chaque clip
from moviepy.video.fx import CrossFadeIn, CrossFadeOut
duree_transition = 0.5
clips_avec_fondu = []
for i, c in enumerate(clips):
    if i > 0:
        c = c.with_effects([CrossFadeIn(duree_transition)])
    clips_avec_fondu.append(c)
final = concatenate_videoclips(clips_avec_fondu, padding=-duree_transition)

```

[vc] Clips à partir d'images – ImageClip / ImageSequenceClip

DESCRIPTION : Créer des clips à partir d'images statiques ou de séquences.
POURQUOI : Diaporamas, animations frame par frame, intros.

FONCTIONS PRINCIPALES :

```

from moviepy import ImageClip, ImageSequenceClip

clip = ImageClip("image.png")           Image statique infinie
clip = ImageClip("image.png",           Image statique de 5 secondes
    duration=5)
clip = ImageClip("image.png",           Utilisé comme masque
    ismask=True)
clip = ImageClip(numpy_array)           Depuis un array NumPy (H, W, 3)

# Séquence d'images
clip = ImageSequenceClip(["f1.png", "f2.png", "f3.png"], fps=24)
clip = ImageSequenceClip("dossier_images/", fps=30)  Depuis un dossier
clip = ImageSequenceClip(list_of_arrays, fps=24)      Depuis des arrays NumPy

```

EXEMPLES :

```

# Diaporama
from moviepy import ImageClip, concatenate_videoclips
images = ["slide1.png", "slide2.png", "slide3.png"]
clips = [ImageClip(img).with_duration(3) for img in images]
diaporama = concatenate_videoclips(clips, method="compose")
diaporama.write_videofile("diaporama.mp4", fps=24)

```

[vc] Texte sur vidéo – TextClip

DESCRIPTION : Générer un clip texte (sous-titres, titres, légendes).
POURQUOI : Incruster du texte animé ou statique sur une vidéo.
CONTEXTE : Nécessite ImageMagick installé sur le système.

FONCTIONS PRINCIPALES :

```

from moviepy import TextClip

txt = TextClip(
    text="Mon titre",           Nom de la police
    font="Arial",              Taille en pixels
    font_size=60,              Couleur du texte
    color="white",              Fond (transparent, black, #RRGGBB)
    bg_color="transparent",     Largeur fixe, hauteur auto
    size=(1920, None),          "label" (une ligne) ou "caption" (retour auto)
    method="caption",           "west", "east", "center"
    align="center",             Couleur du contour
    stroke_color="black",       Épaisseur du contour
    stroke_width=2,             Interlignage
    interline=4,               Durée du clip texte
    duration=5,
)

txt.with_position("center")     Centré sur le fond
txt.with_position(("center", "bottom")) En bas centré
txt.with_start(2).with_end(7)  Visible entre 2 et 7s

```

LISTER LES POLICES DISPONIBLES :

```

TextClip.list("font")           Liste toutes les polices (ImageMagick)

```

```

EXAMPLES      :
from moviepy import VideoFileClip, TextClip, CompositeVideoClip

video = VideoFileClip("video.mp4")
titre = (TextClip(text="Mon Film", font="Arial-Bold",
                  font_size=80, color="white",
                  stroke_color="black", stroke_width=2)
         .with_duration(3)
         .with_position("center")
         .with_effects([FadeIn(0.5), FadeOut(0.5)]))

final = CompositeVideoClip([video, titre])
final.write_videofile("avec_titre.mp4", fps=video.fps)

```

[ac] Audio – AudioFileClip / CompositeAudioClip

DESCRIPTION : Charger, modifier et composer des pistes audio.
POURQUOI : Ajouter musique, voix off, effets sonores, modifier le volume.

— CHARGEMENT —

```
from moviepy import AudioFileClip
```

audio = AudioFileClip("musique.mp3")	Charge un fichier audio
audio = AudioFileClip("son.wav")	WAV, MP3, AAC, OGG, FLAC...
audio = AudioFileClip("video.mp4")	Extrait l'audio d'une vidéo
audio.duration	Durée en secondes
audio.fps	Fréquence d'échantillonnage
audio.nchannels	Nombre de canaux (1=mono, 2=stéréo)

— MODIFIER L'AUDIO —

```
from moviepy.audio.fx import MultiplyVolume, AudioFadeIn, AudioFadeOut, AudioNormalize
```

audio.with_effects([MultiplyVolume(1.5)])	Volume x1.5 (+50%)
audio.with_effects([MultiplyVolume(0.5)])	Volume x0.5 (-50%)
audio.with_effects([AudioFadeIn(2)])	Fondu d'entrée 2s
audio.with_effects([AudioFadeOut(2)])	Fondu de sortie 2s
audio.with_effects([AudioNormalize()])	Normalise le volume
audio.subclipped(10, 30)	Extrait une portion
audio.with_start(5)	Commence à 5s dans la timeline

— MODIFIER L'AUDIO D'UN CLIP VIDÉO —

clip.without_audio()	Supprime la piste audio
clip.with_audio(audio)	Remplace/ajoute l'audio
clip.audio	Accède à la piste audio
clip.audio.with_effects([MultiplyVolume(0.3)])	Baisse le volume

— COMPOSITION AUDIO —

```
from moviepy import CompositeAudioClip
```

```

mix = CompositeAudioClip([
    clip.audio,
    audio_musique.with_effects([MultiplyVolume(0.3)]).with_start(0),
])
clip_final = clip.with_audio(mix)

```

EXAMPLES :

```

# Remplacer l'audio d'une vidéo par une musique
video = VideoFileClip("video.mp4").without_audio()
musique = (AudioFileClip("bg.mp3")
          .subclipped(0, video.duration)
          .with_effects([AudioFadeOut(2)]))
final = video.with_audio(musique)
final.write_videofile("avec_musique.mp4")

# Mixer voix + musique de fond
voix = AudioFileClip("narration.mp3")
musique = AudioFileClip("ambiance.mp3").with_effects([MultiplyVolume(0.2)])
mix = CompositeAudioClip([voix, musique])
video_final = video.with_audio(mix)

```

[vc] Accès aux frames – get_frame / iter_frames

DESCRIPTION : Accéder aux frames individuelles d'un clip.
POURQUOI : Analyse image par image, traitement ML, extraction de frames.

FONCTIONS PRINCIPALES :

clip.get_frame(t)	Frame à l'instant t (array NumPy HxWx3)
clip.get_frame(0)	Première frame
clip.get_frame(clip.duration - 0.001)	Dernière frame
clip.iter_frames()	Itérateur sur toutes les frames
clip.iter_frames(fps=10)	Itérateur à 10 fps
clip.iter_frames(with_times=True)	Itère (temps, frame)
clip.iter_frames(dtype="uint8")	Type de données

EXEMPLES :

```
import numpy as np

# Extraire une frame spécifique
frame = clip.get_frame(5.0) # Frame à 5 secondes
from PIL import Image
Image.fromarray(frame).save("frame_5s.png")

# Analyser toutes les frames
for t, frame in clip.iter_frames(fps=1, with_times=True):
    print(f"t={t:.1f}s : luminosité moyenne = {frame.mean():.1f}")

# Créer un clip depuis une fonction
from moviepy import VideoClip
def make_frame(t):
    # Retourne un array NumPy HxWx3 pour l'instant t
    arr = np.zeros((480, 640, 3), dtype=np.uint8)
    arr[:, :, 0] = int(255 * t / 5) # Canal rouge qui augmente
    return arr

clip_gen = VideoClip(make_frame, duration=5)
```

[vc] Export – write_videofile / write_gif / write_images_sequence

DESCRIPTION : Sauvegarder un clip dans un fichier vidéo, GIF ou séquence d'images.
POURQUOI : Finaliser le montage, exporter pour diffusion.

— EXPORT VIDÉO —

clip.write_videofile("sortie.mp4")	Export MP4 (H.264 + AAC)
clip.write_videofile("sortie.mp4", fps=30, codec="libx264", audio_codec="aac", bitrate="5000k", audio_bitrate="192k", preset="medium", threads=4, ffmpeg_params=["-crf", "18"], logger="bar", temp_audiofile="temp_audio.m4a", remove_temp=True, write_logfile=False,)	Images par seconde Codec vidéo Codec audio Débit vidéo Débit audio Vitesse/qualité (ultrafast→veryslow) Nombre de threads FFmpeg Paramètres FFmpeg arbitraires Barre de progression ("bar" ou None) Fichier audio temporaire Supprime les fichiers temporaires Log FFmpeg dans un fichier

— CODECS COURANTS —

"libx264"	H.264 (MP4, MKV) – le plus compatible
"libx265"	H.265/HEVC – meilleure compression
"libvpx-vp9"	VP9 (WebM) – open source
"mpeg4"	MPEG-4 (AVI)
"rawvideo"	Vidéo brute (non compressée)
"png"	Séquence PNG

— PRESETS H.264 —

"ultrafast"	Encodage très rapide, gros fichier
"superfast", "veryfast", "faster", "fast"	Équilibre vitesse/taille (défaut)
"medium"	Encodage lent, petit fichier
"slow", "slower", "veryslow"	

— EXPORT GIF —

clip.write_gif("animation.gif")	Export GIF
---------------------------------	------------

```
clip.write_gif("animation.gif",
    fps=10,
    program="ffmpeg",
    opt="OptimizePlus",
    fuzz=1)
FPS du GIF
Programme ("ffmpeg" ou "imageio")
Optimisation (imageio)
Seuil de fusion des couleurs
```

— EXPORT SÉQUENCE D'IMAGES —

```
clip.write_images_sequence("frames/frame%04d.png") Séquence PNG
clip.write_images_sequence("frames/frame%04d.jpg",
    fps=5) À 5 fps
```

— EXPORT AUDIO SEUL —

```
clip.audio.write_audiofile("audio.mp3")
clip.audio.write_audiofile("audio.wav",
    fps=44100,
    nbytes=2,
    codec="pcm_s16le")
```

EXEMPLES :

```
# Export haute qualité
clip.write_videofile("hq.mp4",
    fps=30, codec="libx264",
    ffmpeg_params=["-crf", "18", "-pix_fmt", "yuv420p"],
    audio_codec="aac", audio_bitrate="320k")

# Export pour le web (petite taille)
clip.resized(width=1280).write_videofile("web.mp4",
    fps=30, codec="libx264", preset="slow",
    ffmpeg_params=["-crf", "28"])

# GIF court
clip.subclipped(0, 3).resized(width=480).write_gif("preview.gif", fps=12)
```

[vc] Clips spéciaux – ColorClip / VideoClip

DESCRIPTION : Créer des clips synthétiques (fond coloré, généré par code).
 POURQUOI : Fonds, placeholders, animations procédurales.

FONCTIONS PRINCIPALES :

```
from moviepy import ColorClip, VideoClip

ColorClip(size=(1920,1080),
    color=(0,0,0),
    duration=5) Fond noir 5 secondes
ColorClip(size=(1280,720),
    color=(255,255,255),
    duration=3) Fond blanc
ColorClip(size=(1920,1080),
    color=(30,30,30),
    duration=clip.duration) Fond pour composition
```

```
# Clip généré par une fonction Python
def make_frame(t):
    import numpy as np
    frame = np.zeros((480, 640, 3), dtype=np.uint8)
    # Animation : cercle qui se déplace
    cx = int(320 + 200 * np.cos(2 * np.pi * t))
    cy = int(240 + 150 * np.sin(2 * np.pi * t))
    import cv2
    cv2.circle(frame, (cx, cy), 30, (255, 100, 0), -1)
    return frame

clip_gen = VideoClip(make_frame, duration=3)
clip_gen.fps = 30
```

EXEMPLES :

```
# Titre sur fond noir
from moviepy import ColorClip, TextClip, CompositeVideoClip

fond = ColorClip((1920, 1080), color=(0,0,0), duration=5)
titre = (TextClip(text="CHAPITRE 1", font="Arial-Bold",
    font_size=100, color="white")
    .with_duration(5))
```



```

        .with_position("center"))
intro = CompositeVideoClip([fond, titre])

```

[vc] Masques et opacité – with_opacity / with_mask

DESCRIPTION : Contrôler la transparence et appliquer des masques.
 POURQUOI : Incrustations semi-transparentes, transitions masquées.

FONCTIONS PRINCIPALES :

clip.with_opacity(0.5)	50% de transparence
clip.with_opacity(0.0)	Invisible
clip.with_opacity(1.0)	Opaque (défaut)
clip.add_mask()	Ajoute un masque vide (tout opaque)
clip.mask	Accède au masque (VideoClip niveaux de gris)

```

# Masque depuis une image
from moviepy import ImageClip
masque = ImageClip("masque.png", ismask=True)
clip_masque = clip.with_mask(masque)

```

EXEMPLES :

```

# Logo semi-transparent
logo = (ImageClip("logo.png")
        .with_duration(video.duration)
        .resized(height=80)
        .with_opacity(0.4)
        .with_position(("right", "bottom")))
final = CompositeVideoClip([video, logo])

```

[vc] Recettes avancées – patterns courants

— SOUS-TITRES AUTOMATIQUES —

```

def ajouter_sous_titre(video, texte, debut, fin):
    txt = (TextClip(text=texte, font="Arial", font_size=40,
                    color="white", stroke_color="black", stroke_width=1,
                    method="caption", size=(video.w * 0.8, None))
           .with_start(debut).with_end(fin)
           .with_position(("center", "bottom")))
    return txt

sous_titres = [("Bonjour !", 1, 3), ("Bienvenue.", 3.5, 6)]
clips_txt = [ajouter_sous_titre(video, t, d, f) for t, d, f in sous_titres]
final = CompositeVideoClip([video] + clips_txt)

```

— SPLIT SCREEN —

```

gauche = VideoFileClip("a.mp4").resized((960, 1080)).with_position((0, 0))
droite = VideoFileClip("b.mp4").resized((960, 1080)).with_position((960, 0))
final = CompositeVideoClip([gauche, droite], size=(1920, 1080))

```

— EXTRACTION DE TOUTES LES FRAMES EN PNG —

```

import os
os.makedirs("frames", exist_ok=True)
clip.write_images_sequence("frames/frame%05d.png", fps=clip.fps)

```

— CONVERTIR EN NOIR ET BLANC ET EXPORTER —

```

from moviepy.video.fx import BlackAndWhite
(VideoFileClip("couleur.mp4")
 .with_effects([BlackAndWhite()])
 .write_videofile("nb.mp4"))

```

— CRÉER UN GIF DEPUIS UNE VIDÉO —

```

(VideoFileClip("video.mp4")
 .subclipped(5, 10)
 .resized(width=480)
 .write_gif("extrait.gif", fps=12))

```

— STABILISER LE VOLUME AUDIO —

```

from moviepy.audio.fx import AudioNormalize
(VideoFileClip("video.mp4")

```

```
.with_audio(
    VideoFileClip("video.mp4").audio
    .with_effects([AudioNormalize()])
)
.write_videofile("normalise.mp4"))
```

RÉCAPITULATIF MOVIEPY

TÂCHE	MÉTHODE / CLASSE
Charger une vidéo	VideoFileClip("fichier.mp4")
Charger une image	ImageClip("image.png")
Charger de l'audio	AudioFileClip("son.mp3")
Fond coloré	ColorClip(size, color, duration)
Texte	TextClip(text, font, font_size, color)
Clip généré	VideoClip(make_frame, duration)
Découper	clip.subclipped(debut, fin)
Redimensionner	clip.resized(width=1280)
Recadrer	clip.cropped(x1, y1, x2, y2)
Rotation	clip.rotated(90)
Vitesse	clip.with_effects([MultiplySpeed(x)])
Fondu entrée/sortie	clip.with_effects([FadeIn(t), FadeOut(t)])
Noir et blanc	clip.with_effects([BlackAndWhite()])
Lecture à l'envers	clip.with_effects([TimeMirror()])
Superposer des clips	CompositeVideoClip([fond, overlay])
Positionner un overlay	overlay.with_position(("center", "bottom"))
Concaténer	concatenate_videoclips([c1, c2, c3])
Opacité	clip.with_opacity(0.5)
Accès frame NumPy	clip.get_frame(t)
Transformation frame par frame	clip.image_transform(fn) / clip.transform(fn)
Supprimer l'audio	clip.without_audio()
Remplacer l'audio	clip.with_audio(audio_clip)
Modifier le volume	audio.with_effects([MultiplyVolume(0.5)])
Mélanger des audios	CompositeAudioClip([a1, a2])
Export MP4	clip.write_videofile("out.mp4", fps=30)
Export GIF	clip.write_gif("out.gif", fps=10)
Export audio	clip.audio.write_audiofile("out.mp3")
Export séquence images	clip.write_images_sequence("f%04d.png")

— VERSIONS —

MoviePy v1.x	API avec .fx(), .set_*, .fl_image()	(ancienne API)
MoviePy v2.x	API avec .with_effects([]), .with_*	(API moderne ☐)

Cette bible couvre MoviePy v2.x (API moderne recommandée)

— DÉPENDANCES —

FFmpeg	Requis – encodage/décodage vidéo
ImageMagick	Requis pour TextClip
NumPy	Requis – manipulation des frames
Pillow	Optionnel – manipulation d'images
opencv-python	Optionnel – traitements avancés de frames
